ECEN 375
Computer Architecture and Organization
Unit 3 - Architecture: Assembly - High-Level Constructs

Dr. Daniel Limbrick

Department of Electrical and Computer Engineering
North Carolina Agricultural and Technical State University

Spring 2022



Topics

- **Administrative Items:**
  - Reading Assignments:
    - Textbook: Chapter 6, Sections 3 and 4
  - Homework Assignments:
    - Textbook Problems: 6.18, 6.22, 6.24, 6.26 (due Sunday, February 6)
  - Lab Assignments:
    - Lab 2: Due Sunday, February 6
  - Exam 1: ~~Sunday, February 13~~ Sunday, February 20
- **Loops**
- **Conditional Execution (e.g., If/Else)**
- **Functions**
- **The Stack**

Programming Building Blocks

- Data-processing Instructions
- Conditional Execution
- Branches
- **High-level Constructs:**
  - **if/else statements**
  - **for loops**
  - **while loops**
  - arrays
  - function calls

Digital Design and Computer Architecture: ARM® Edition © 2015

C Code

```
if (i == j)
  f = g + h;
```

```
f = f - i;
```

C Code

```
if (i == j)
  f = g + h;

f = f - i;
```

ARM Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

  CMP R3, R4        ; set flags with R3-R4
  BNE L1            ; if i!=j, skip if block
  ADD R0, R1, R2    ; f = g + h

L1
  SUB R0, R0, R3    ; f = f - i
```

C Code

```
if (i == j)
  f = g + h;

f = f - i;
```

ARM Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

  CMP R3, R4        ; set flags with R3-R4
  BNE L1            ; if i!=j, skip if block
  ADD R0, R1, R2    ; f = g + h

L1
  SUB R0, R0, R3    ; f = f - i
```

Assembly tests opposite case (`i != j`) of high-level code (`i == j`)

C Code

```
if (i == j)
  f = g + h;
f = f - i;
```

ARM Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

 CMP   R3, R4       ; set flags with R3-R4
 ADDEQ R0, R1, R2   ; if (i==j) f = g + h
 SUB   R0, R0, R3   ; f = f - i
```

Original

```
 CMP R3, R4
 BNE L1
 ADD R0, R1, R2
L1
 SUB R0, R0, R3
```

Alternate Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

 CMP   R3, R4       ; set flags with R3-R4
 ADDEQ R0, R1, R2   ; if (i==j) f = g + h
 SUB   R0, R0, R3   ; f = f - i
```

if Statement: Alternate Code

Original

```
 CMP R3, R4
 BNE L1
 ADD R0, R1, R2
L1
 SUB R0, R0, R3
```

Alternate Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

 CMP   R3, R4       ; set flags with R3-R4
 ADDEQ R0, R1, R2   ; if (i==j) f = g + h
 SUB   R0, R0, R3   ; f = f - i
```

Useful for **short** conditional blocks of code

C Code

```
if (i == j)
  f = g + h;


else
  f = f - i;
```

ARM Assembly Code

C Code

```
if (i == j)
  f = g + h;

else
  f = f - i;
```

ARM Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

 CMP R3, R4       ; set flags with R3-R4
 BNE L1           ; if i!=j, skip if block
 ADD R0, R1, R2   ; f = g + h
 B   L2           ; branch past else block
L1
 SUB R0, R0, R3   ; f = f - i
L2
```

C Code

```
if (i == j)
  f = g + h;
else
   f = f - i;
```

ARM Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

   CMP   R3, R4       ; set flags with R3-R4
   ADDEQ R0, R1, R2   ; if (i==j) f = g + h
   SUBNE R0, R0, R3   ; else f = f - i
```

Original

```
 CMP R3, R4
 BNE L1
 ADD R0, R1, R2
 B   L2
L1
 SUB R0, R0, R3
L2
```

Alternate Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

 CMP   R3, R4       ; set flags with R3-R4
 ADDEQ R0, R1, R2   ; if (i==j) f = g + h

 SUBNE R0, R0, R3   ; else f = f - i
```

C Code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;


while (pow != 128) {

  pow = pow * 2;
  x = x + 1;
}
```

ARM Assembly Code

C Code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;


while (pow != 128) {

  pow = pow * 2;
  x = x + 1;
}
```

ARM Assembly Code

```
; R0 = pow, R1 = x
  MOV   R0, #1          ; pow = 1
  MOV   R1, #0          ; x = 0

WHILE
  CMP R0, #128          ; R0-128
  BEQ DONE              ; if (pow==128)
                        ; exit loop
  LSL R0, R0, #1        ; pow=pow*2
  ADD R1, R1, #1        ; x=x+1
  B   WHILE             ; repeat loop

DONE
```

while Loops

C Code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;

while (pow != 128) {
  pow = pow * 2;
  x = x + 1;
}
```

ARM Assembly Code

```
; R0 = pow, R1 = x
  MOV   R0, #1          ; pow = 1
  MOV   R1, #0          ; x = 0

WHILE
  CMP R0, #128          ; R0-128
  BEQ DONE              ; if (pow==128)
                        ; exit loop
  LSL R0, R0, #1        ; pow=pow*2
  ADD R1, R1, #1        ; x=x+1
  B   WHILE             ; repeat loop

DONE
```

Assembly tests for the opposite case (`pow == 128`) of the C code (`pow != 128`).

for Loops

```
for (initialization; condition; loop operation)
   statement
```

- `initialization:` executes before the loop begins
- `condition:` is tested at the beginning of each iteration
- `loop operation:` executes at the end of each iteration
- `statement:` executes each time the condition is met

C Code

```
// adds numbers from 1-9
int sum = 0


for (i=1; i!=10; i=i+1)
   sum = sum + i;
```

ARM Assembly Code

C Code

```
// adds numbers from 1-9
int sum = 0


for (i=1; i!=10; i=i+1)
   sum = sum + i;
```

ARM Assembly Code

```
; R0 = i, R1 = sum
  MOV   R0, #1          ; i = 1
  MOV   R1, #0          ; sum = 0

FOR
  CMP R0, #10           ; R0-10
  BEQ DONE              ; if (i==10)
                        ; exit loop
  ADD R1, R1, R0        ; sum=sum + i
  ADD R0, R0, #1        ; i = i + 1
  B   FOR               ; repeat loop

DONE
```

In ARM, decremented loop variables are more efficient

for Loops: Decremented Loops

In ARM, decremented loop variables are more efficient

C Code

```
// adds numbers from 1-9
int sum = 0


for (i=9; i!=0; i=i-1)
   sum = sum + i;
```

ARM Assembly Code

```
; R0 = i, R1 = sum
  MOV   R0, #9          ; i = 9
  MOV   R1, #0          ; sum = 0

FOR
  ADD   R1, R1, R0      ; sum=sum + i
  SUBS  R0, R0, #1      ; i = i - 1
                        ; and set flags
  BNE   FOR             ; if (i!=0)
                        ; repeat loop
```

for Loops: Decremented Loops

In ARM, decremented loop variables are more efficient

C Code

```
// adds numbers from 1-9
int sum = 0


for (i=9; i!=0; i=i-1)
   sum = sum + i;
```

ARM Assembly Code

```
; R0 = i, R1 = sum
  MOV   R0, #9          ; i = 9
  MOV   R1, #0          ; sum = 0

FOR
  ADD   R1, R1, R0      ; sum=sum + i
  SUBS  R0, R0, #1      ; i = i - 1
                        ; and set flags
  BNE   FOR             ; if (i!=0)
                        ; repeat loop
```

Saves 2 instructions per iteration:

- Decrement loop variable & compare: `SUBS R0, R0, #1`
- Only 1 branch – instead of 2

Programming Building Blocks

- Data-processing Instructions
- Conditional Execution
- Branches
- **High-level Constructs:**
  - if/else statements
  - for loops
  - while loops
  - **arrays**
  - function calls

Arrays

- Access large amounts of similar data
  - **Index:** access to each element
  - **Size:** number of elements

- 5-element array
  - **Base address** = 0x14000000 (address of first element, scores[0])
  - Array elements accessed relative to base address



Accessing Arrays

C Code

```
int array[5];
array[0] = array[0] * 8;
array[1] = array[1] * 8;
```

ARM Assembly Code

```
; R0 = array base address
```

Accessing Arrays

C Code

```
int array[5];
array[0] = array[0] * 8;
array[1] = array[1] * 8;
```

ARM Assembly Code

```
; R0 = array base address
  MOV R0, #0x60000000          ; R0 = 0x60000000

  LDR R1, [R0]                 ; R1 = array[0]
  LSL R1, R1, 3                ; R1 = R1 << 3 = R1*8
  STR R1, [R0]                 ; array[0] = R1

  LDR R1, [R0, #4]             ; R1 = array[1]
  LSL R1, R1, 3                ; R1 = R1 << 3 = R1*8
  STR R1, [R0, #4]             ; array[1] = R1
```

Arrays using for Loops

C Code

```
int array[200];
int i;

for (i=199; i >= 0; i = i - 1)
     array[i] = array[i] * 8;
```

ARM Assembly Code

```
; R0 = array base address, R1 = i
```

Arrays using for Loops

C Code

```
int array[200];
int i;

for (i=199; i >= 0; i = i - 1)
     array[i] = array[i] * 8;
```

ARM Assembly Code

```
; R0 = array base address, R1 = i
  MOV R0, 0x60000000
  MOV R1, #199

FOR
  LDR  R2, [R0, R1, LSL #2]      ; R2 = array(i)
  LSL  R2, R2, #3                ; R2 = R2<<3 = R3*8
  STR  R2, [R0, R1, LSL #2]      ; array(i) = R2
  SUBS R0, R0, #1                ; i = i - 1
                                 ; and set flags
  BPL  FOR                       ; if (i>=0) repeat loop
```

Digital Design and Computer Architecture: ARM® Edition © 2015

Programming Building Blocks

- Data-processing Instructions
- Conditional Execution
- Branches
- **High-level Constructs:**
  - if/else statements
  - for loops
  - while loops
  - arrays
  - **function calls**

- **Caller:** calling function (in this case, `main`)
- **Callee:** called function (in this case, `sum`)

C Code

```
void main()
{
  int y;
  y = sum(42, 7);
  ...
}

int sum(int a, int b)
{
  return (a + b);
}
```

Function Conventions

- **Caller:**
  - passes **arguments** to callee
  - jumps to callee

Function Conventions

- **Caller:**
  - passes **arguments** to callee
  - jumps to callee
- **Callee:**
  - **performs** the function
  - **returns** result to caller
  - **returns** to point of call
  - **must not overwrite** registers or memory needed by caller

ARM Function Conventions

- **Call Function:** branch and link

  `BL`
- **Return** from function: move the link register to PC: `MOV PC, LR`
- **Arguments:** `R0-R3`
- **Return value:** `R0`

C Code

```
int main() {
  simple();
  a = b + c;
}

void simple() {
  return;
}
```

ARM Assembly Code

```
0x00000200 MAIN     BL  SIMPLE
0x00000204          ADD R4, R5, R6
...

0x00401020 SIMPLE   MOV PC, LR
```

C Code

```
int main() {
  simple();
  a = b + c;
}

void simple() {
  return;
}
```

ARM Assembly Code

```
0x00000200 MAIN     BL  SIMPLE
0x00000204          ADD R4, R5, R6
...

0x00401020 SIMPLE   MOV PC, LR
```

`void` means that `simple` doesn't return a value

C Code

```
int main() {
  simple();
  a = b + c;
}

void simple() {
  return;
}
```

ARM Assembly Code

```
0x00000200 MAIN     BL  SIMPLE
0x00000204          ADD R4, R5, R6
...

0x00401020 SIMPLE   MOV PC, LR
```

BL branches to SIMPLE
LR = PC + 4 = 0x00000204

MOV PC, LR makes PC = LR
(the next instruction executed is at 0x00000204)

Input Arguments and Return Value

ARM conventions:

- Argument values: `R0` - `R3`
- Return value: `R0`

C Code

```
int main()
{
  int y;
  ...
  y = diffofsums(2, 3, 4, 5);  // 4 arguments
  ...
}

int diffofsums(int f, int g, int h, int i)
{
  int result;
  result = (f + g) - (h + i);
  return result;                // return value
}
```

Input Arguments and Return Value

ARM Assembly Code

```
; R4 = y
MAIN
  ...
  MOV R0, #2          ; argument 0 = 2
  MOV R1, #3          ; argument 1 = 3
  MOV R2, #4          ; argument 2 = 4
  MOV R3, #5          ; argument 3 = 5
  BL DIFFOFSUMS       ; call function
  MOV R4, R0          ; y = returned value
  ...
; R4 = result
DIFFOFSUMS
  ADD R8, R0, R1      ; R8 = f + g
  ADD R9, R2, R3      ; R9 = h + i
  SUB R4, R8, R9      ; result = (f + g) - (h + i)
  MOV R0, R4          ; put return value in R0
  MOV PC, LR          ; return to caller
```

ARM Assembly Code

```
; R4 = result
DIFFOFSUMS
  ADD R8, R0, R1      ; R8 = f + g
  ADD R9, R2, R3      ; R9 = h + i
  SUB R4, R8, R9      ; result = (f + g) - (h + i)
  MOV R0, R4          ; put return value in R0
  MOV PC, LR          ; return to caller
```

- `diffofsums` overwrote 3 registers: `R4`, `R8`, `R9`
- `diffofsums` can use *stack* to temporarily store registers

The Stack

- Memory used to temporarily save variables
- Like stack of dishes, last-in-first-out (LIFO) queue
- ***Expands***: uses more memory when more space needed
- ***Contracts***: uses less memory when the space no longer needed



- Grows down (from higher to lower memory addresses)
- Stack pointer: `SP` points to top of the stack



Stack expands by 2 words

- Called functions must have no unintended side effects
- But `diffofsums` overwrites 3 registers: `R4`, `R8`, `R9`

ARM Assembly Code

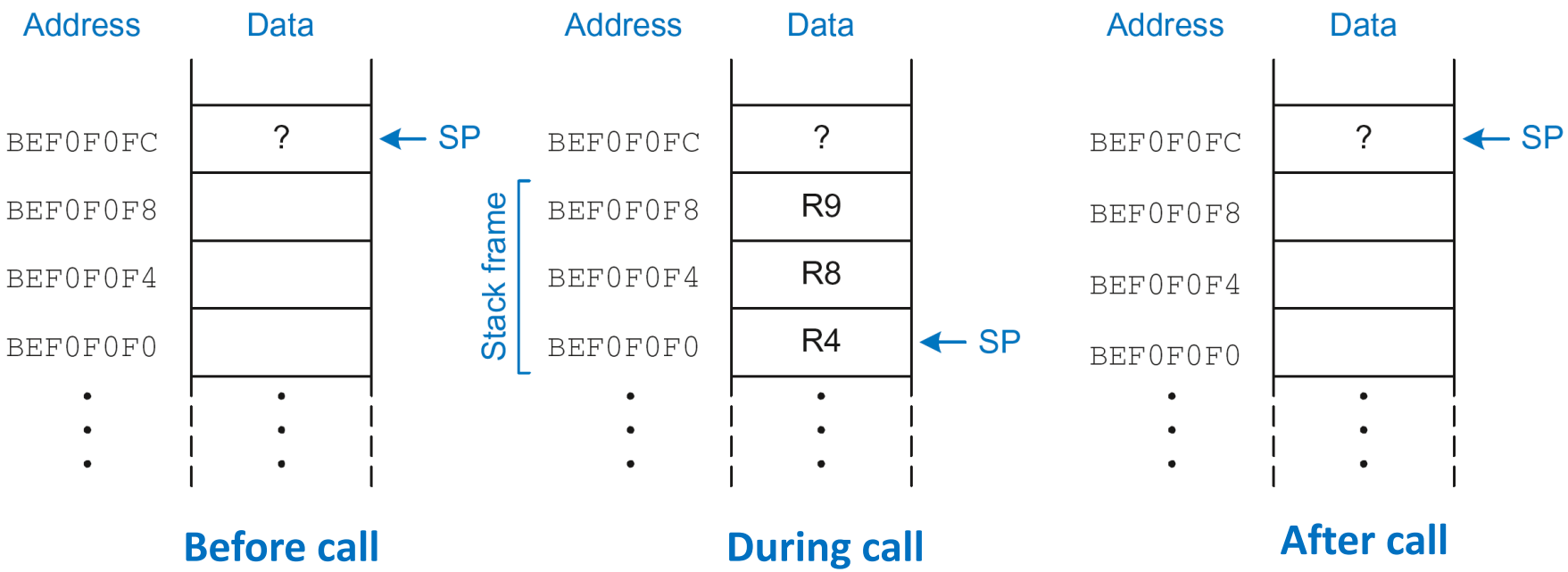
```
; R4 = result
DIFFOFSUMS
  ADD R8, R0, R1      ; R8 = f + g
  ADD R9, R2, R3      ; R9 = h + i
  SUB R4, R8, R9      ; result = (f + g) - (h + i)
  MOV R0, R4          ; put return value in R0
  MOV PC, LR          ; return to caller
```

ARM Assembly Code

```
; R2 = result
DIFFOFSUMS
  SUB SP, SP, #12     ; make space on stack for 3 registers
  STR R4, [SP, #8]    ; save R4 on stack
  STR R8, [SP, #4]    ; save R8 on stack
  STR R9, [SP]        ; save R9 on stack
  ADD R8, R0, R1      ; R8 = f + g
  ADD R9, R2, R3      ; R9 = h + i
  SUB R4, R8, R9      ; result = (f + g) - (h + i)
  MOV R0, R4          ; put return value in R0
  LDR R9, [SP]        ; restore R9 from stack
  LDR R8, [SP, #4]    ; restore R8 from stack
  LDR R4, [SP, #8]    ; restore R4 from stack
  ADD SP, SP, #12     ; deallocate stack space
  MOV PC, LR          ; return to caller
```

Digital Design and Computer Architecture: ARM® Edition © 2015

The Stack during diffofsums Call



Registers

Preserved *Callee-Saved*	Nonpreserved *Caller-Saved*
R4-R11	R12
R14 (LR)	R0-R3
R13 (SP)	CPSR
stack above SP	stack below SP

ARM Assembly Code

```
; R2 = result
DIFFOFSUMS
  STR R4, [SP, #-4]! ; save R4 on stack
  ADD R8, R0, R1     ; R8 = f + g
  ADD R9, R2, R3     ; R9 = h + i
  SUB R4, R8, R9     ; result = (f + g) - (h + i)
  MOV R0, R4         ; put return value in R0
  LDR R4, [SP], #4   ; restore R4 from stack
  MOV PC, LR         ; return to caller
```

ARM Assembly Code

```
; R2 = result
DIFFOFSUMS
  STR R4, [SP, #-4]! ; save R4 on stack
  ADD R8, R0, R1     ; R8 = f + g
  ADD R9, R2, R3     ; R9 = h + i
  SUB R4, R8, R9     ; result = (f + g) - (h + i)
  MOV R0, R4         ; put return value in R0
  LDR R4, [SP], #4   ; restore R4 from stack
  MOV PC, LR         ; return to caller
```

Notice code optimization for expanding/contracting stack

ARM Assembly Code

```
STR LR, [SP, #-4]!      ; store LR on stack
BL  PROC2               ; call another function
...
LDR LR, [SP], #4        ; restore LR from stack
jr  $ra                 ; return to caller
```

Nonleaf Function Example

C Code

```
int f1(int a, int b) {
  int i, x;
  x = (a + b)*(a - b);
  for (i=0; i<a; i++)
    x = x + f2(b+i);
  return x;
}
int f2(int p) {
  int r;
  r = p + 5;
  return r + p;
}
```

Nonleaf Function Example

C Code

```
int f1(int a, int b) {
  int i, x;
  x = (a + b)*(a - b);
  for (i=0; i<a; i++)
    x = x + f2(b+i);
  return x;
}
int f2(int p) {
  int r;
  r = p + 5;
  return r + p;
}
```

ARM Assembly Code

```
; R0=a, R1=b, R4=i, R5=x
F1
  PUSH {R4,  R5, LR}
  ADD   R5,  R0, R1
  SUB   R12, R0, R1
  MUL   R5,  R5, R12
  MOV   R4,  #0
FOR
  CMP   R4, R0
  BGE   RETURN
  PUSH {R0, R1}
  ADD   R0, R1, R4
  BL    F2
  ADD   R5, R5, R0
  POP  {R0, R1}
  ADD   R4, R4, #1
  B     FOR
RETURN
  MOV   R0, R5
  POP  {R4, R5, LR}
  MOV   PC, LR
```

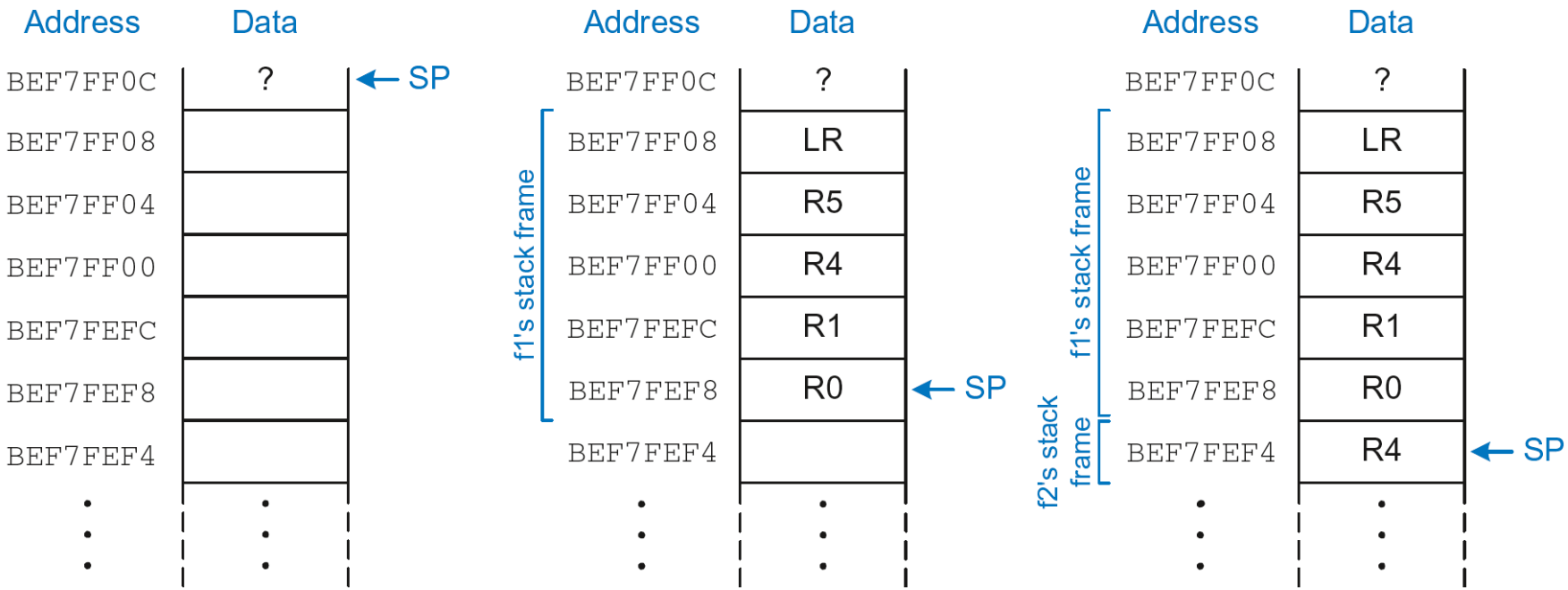
```
; R0=p, R4=r
F2
  PUSH {R4}
  ADD   R4, R0, 5
  ADD   R0, R4, R0
  POP  {R4}
  MOV   PC, LR
```

ARM Assembly Code

```
; R0=a, R1=b, R4=i, R5=x
F1
  PUSH {R4,  R5, LR} ; save regs
  ADD   R5,  R0, R1  ; x = (a+b)
  SUB   R12, R0, R1  ; temp = (a-b)
  MUL   R5,  R5, R12 ; x = x*temp
  MOV   R4,  #0      ; i = 0
FOR
  CMP   R4, R0       ; i < a?
  BGE   RETURN       ; no: exit loop
  PUSH {R0, R1}      ; save regs
  ADD   R0, R1, R4   ; arg is b+i
  BL    F2           ; call f2(b+i)
  ADD   R5, R5, R0   ; x = x+f2(b+i)
  POP  {R0, R1}      ; restore regs
  ADD   R4, R4, #1   ; i++
  B     FOR          ; repeat loop
RETURN
  MOV   R0, R5       ; return x
  POP  {R4, R5, LR}  ; restore regs
  MOV   PC, LR       ; return
```

```
; R0=p, R4=r
F2
  PUSH {R4}          ; save regs
  ADD   R4, R0, 5    ; r = p+5
  ADD   R0, R4, R0   ; return r+p
  POP  {R4}          ; restore regs
  MOV   PC, LR       ; return
```

Stack during Nonleaf Function



C Code

```
int factorial(int n) {
  if (n <= 1)
    return 1;
  else
    return (n * factorial(n-1));
}
```

ARM Assembly Code

```
0x94  FACTORIAL   STR R0, [SP, #-4]!    ;store R0 on stack
0x98              STR LR, [SP, #-4]!    ;store LR on stack
0x9C              CMP R0, #2            ;set flags with R0-2
0xA0              BHS ELSE              ;if (r0>=2) branch to else
0xA4              MOV R0, #1            ; otherwise return 1
0xA8              ADD SP, SP, #8        ; restore SP 1
0xAC              MOV PC, LR            ; return
0xB0  ELSE        SUB R0, R0, #1        ; n = n - 1
0xB4              BL  FACTORIAL         ; recursive call
0xB8              LDR LR, [SP], #4      ; restore LR
0xBC              LDR R1, [SP], #4      ; restore R0 (n) into R1
0xC0              MUL R0, R1, R0        ; R0 = n*factorial(n-1)
0xC4              MOV PC, LR            ; return
```

Recursive Function Call

C Code

```
int factorial(int n) {

  if (n <= 1)
    return 1;



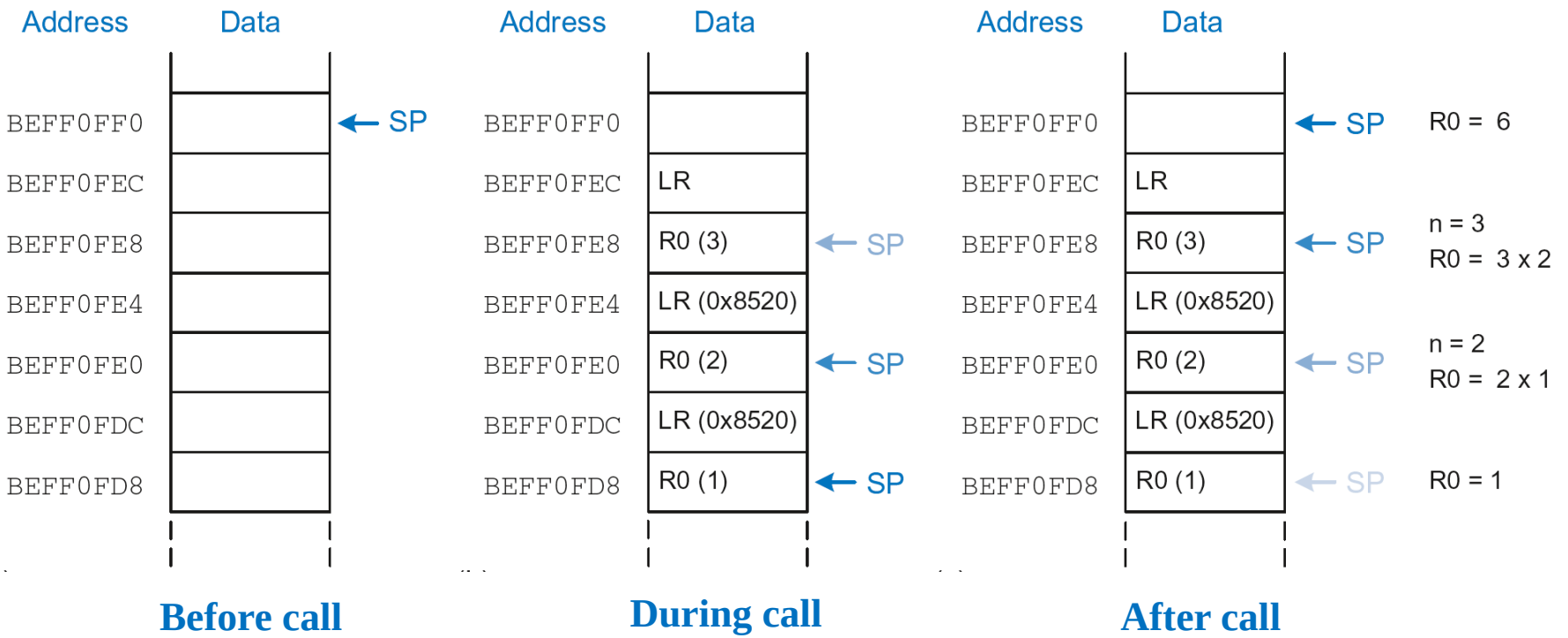
  else
    return (n * factorial(n-1));
}
```

ARM Assembly Code

```
0x94 FACTORIAL  STR R0, [SP, #-4]!
0x98            STR LR, [SP, #-4]!
0x9C            CMP R0, #2
0xA0            BHS ELSE
0xA4            MOV R0, #1
0xA8            ADD SP, SP, #8
0xAC            MOV PC, LR
0xB0 ELSE       SUB R0, R0, #1
0xB4            BL  FACTORIAL
0xB8            LDR LR, [SP], #4
0xBC            LDR R1, [SP], #4
0xC0            MUL R0, R1, R0
0xC4            MOV PC, LR
```

Digital Design and Computer Architecture: ARM® Edition © 2015

Stack during Recursive Call



Function Call Summary

- **Caller**
  - Puts arguments in `R0-R3`
  - Saves any needed registers (`LR`, maybe `R0-R3, R8-R12`)
  - Calls function: `BL CALLEE`
  - Restores registers
  - Looks for result in `R0`
- **Callee**
  - Saves registers that might be disturbed `(R4-R7)`
  - Performs function
  - Puts result in `R0`
  - Restores registers
  - Returns: `MOV PC, LR`

Digital Design and Computer Architecture: ARM® Edition © 2015